

Task 1: Build a RESTful API (Node.js + Express + MongoDB)

Internship

Alfido Tech – MEAN Stack Developer Internship

Objective

To build a RESTful CRUD API using Node.js, Express.js, and MongoDB with Mongoose for data persistence and validation.

Project Overview

A Task Management API was developed to perform Create, Read, Update, and Delete (CRUD) operations on tasks stored in MongoDB Atlas.

Each task contains:

- Title
 - Description
 - Status
 - Priority
 - Created Date
 - Updated Date
-

Technologies Used

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- Postman

- Dotenv
 - Morgan
 - CORS
 - Nodemon
-

Project Structure

alfido-mean-stack/

config/

- db.js

controllers/

- taskController.js

models/

- Task.js

routes/

- taskRoutes.js

.env .env.example server.js

package.json

Installation Commands

npm init -y

npm install express mongoose dotenv cors morgan

npm install --save-dev nodemon

Environment Variables

PORT=3000

MONGO_URI=

MongoDB Connection

The application connects to MongoDB Atlas using Mongoose.

File: config/db.js

Purpose:

- Establish database connection
- Handle connection errors
- Export database connection function

[Screenshot: MongoDB Atlas Cluster]

Task Schema

File: models/Task.js

Fields:

1. title (Required)
2. description (Required)
3. status
 - pending
 - completed
4. priority
 - low
 - medium

- high

Validation was implemented using Mongoose schema validation.

[Screenshot: Task Schema Code]

API Endpoints

1. Create Task

POST /api/tasks

Request Body:

```
{ "title": "Build REST API", "description": "Complete internship task", "priority": "high" }
```

Response: 201 Created

[Screenshot: POST Request and Response]

2. Get All Tasks

GET /api/tasks

Response: 200 OK

[Screenshot: GET All Tasks]

3. Get Single Task

GET /api/tasks/:id

Response: 200 OK

[Screenshot: GET Single Task]

4. Update Task

PUT /api/tasks/:id

Request Body:

```
{ "status": "completed" }
```

Response: 200 OK

[Screenshot: PUT Request]

5. Delete Task

DELETE /api/tasks/:id

Response: 200 OK

[Screenshot: DELETE Request]

Error Handling

Implemented using try-catch blocks.

Handled:

- Invalid requests
- Missing task IDs
- MongoDB connection errors
- Validation errors

[Screenshot: Error Handling Code]

Logging

Morgan middleware was used for request logging.

Example:

GET /api/tasks 200

POST /api/tasks 201

[Screenshot: Terminal Logs]

Results

Successfully implemented a RESTful CRUD API using Node.js, Express.js, and MongoDB.

Features achieved:

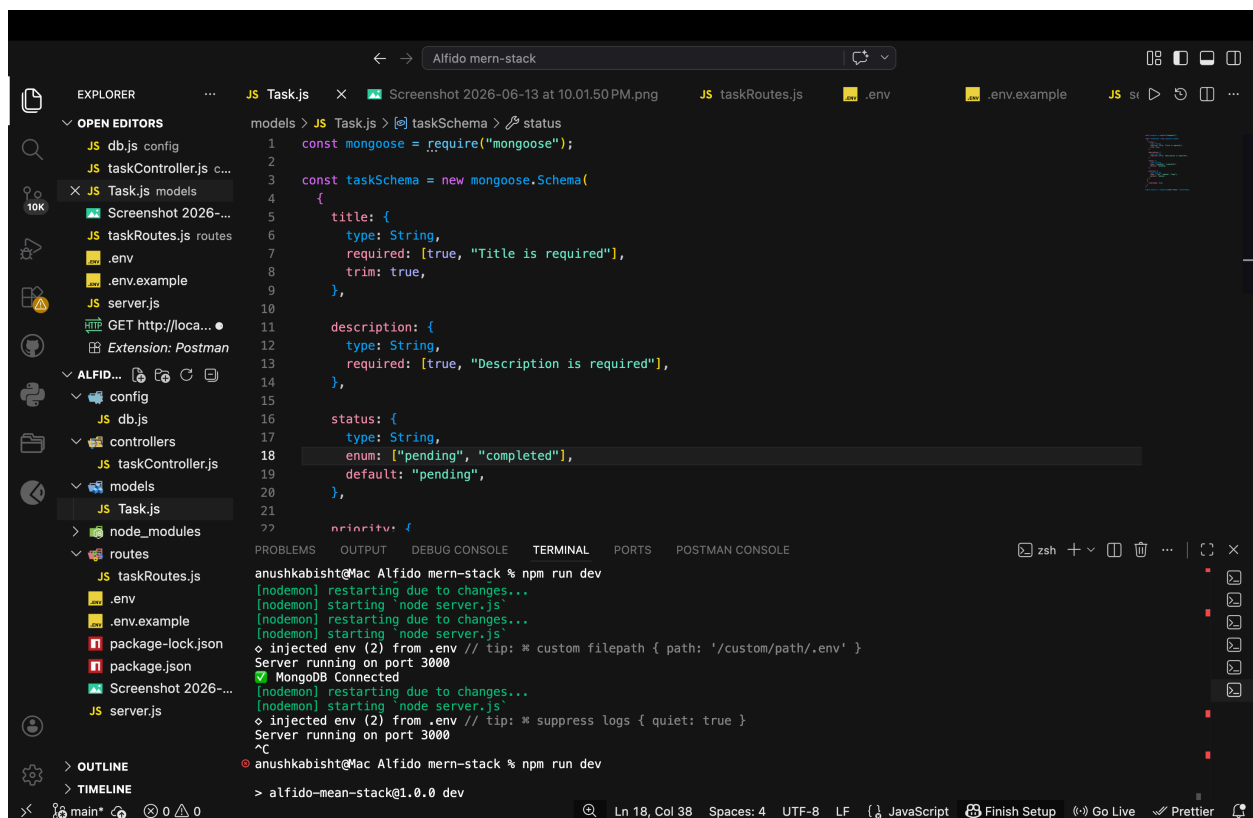
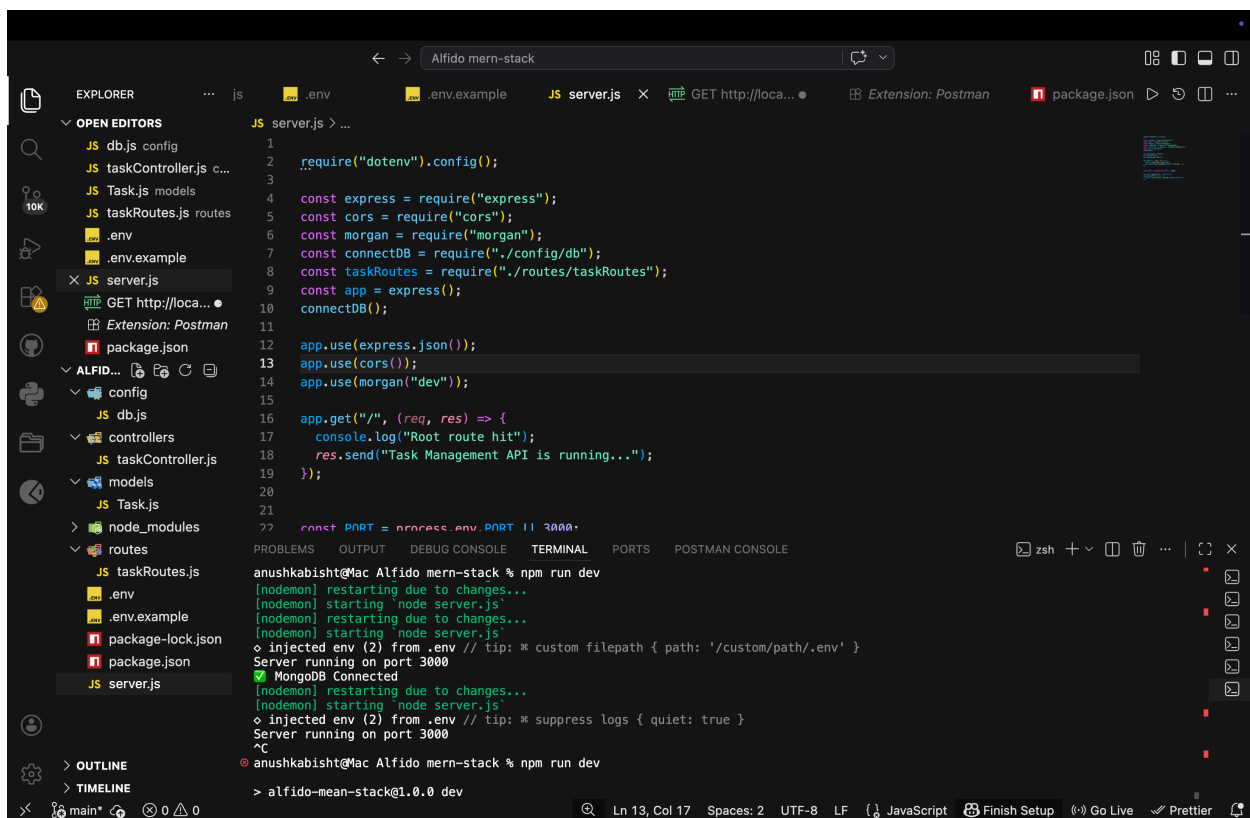
- CRUD Operations
 - MongoDB Integration
 - Mongoose Validation
 - Error Handling
 - Logging
 - Environment Variables
-

GitHub Repository

Repository Link:

Conclusion

The project successfully demonstrates backend development concepts including REST API design, MongoDB integration, CRUD operations, validation, logging, and error handling using the MEAN stack ecosystem.



MongoDB Atlas

Project Overview

DATABASE

Clusters

Search & Vector Search

Data Explorer

Backup

STREAMING DATA

Stream Processing

Triggers

SERVICES

AI Models

Migration

Data Federation

Visualization

SECURITY

Security Quickstart

Project Identity & Access

Database & Network Access

Activity Feed

ORGANIZATION

anu's Org - 2026-06-13

PROJECT

Project 0

?

Clusters

Find a database deployment...

Edit Config

Create

Load sample datasets to Cluster0.

Atlas provides sample data you can load into your Atlas clusters. You can use this data to quickly get started exploring with data in MongoDB.

Load sample dataset

Dismiss

Cluster0

Connect

View Monitoring

Browse Collections

...

FREE

Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Explore Charts

Dismiss

R 0

W

Last 52 minutes

0.004%

Connections

0

Last 52 minutes

9.0

In 0.00 B/s

Out 3.95 B/s

Last 52 minutes

673.05 B/s

Data Size

36.16 KB / 512.00 MB

Last 52 minutes

512.00 MB

VERSION

8.0.26

REGION

AWS / Mumbai (ap-south-1)

TYPE

Replica Set - 3 nodes

BACKUPS

Inactive

LINKED APP SERVICES

None Linked

ATLAS SQL

Connect

ATLAS SEARCH

Create Index

+ Add Tag

Alert System Status: All Good

Alfido mern-stack

EXPLORER

OPEN EDITORS

JS db.js config

JS taskController.js c...

JS Task.js models

Screenshot 2026-...

JS taskRoutes.js routes

.env

.env.example

JS server.js

GET http://loca...

Extension: Postman

package.json

ALFIDO MERN-STACK

config

JS db.js

controllers

JS taskController.js

models

JS Task.js

node_modules

routes

JS taskRoutes.js

.env

.env.example

package-lock.json

package.json

Screenshot 2026-...

JS server.js

OUTLINE

TIMELINE

main*

Ln 11, Col 1

Spaces: 2

UTF-8

LF

JavaScript

Finish Setup

Go Live

Prettier

```
1
2 require("dotenv").config();
3
4 const express = require("express");
5 const cors = require("cors");
6 const morgan = require("morgan");
7 const connectDB = require("../config/db");
8 const taskRoutes = require("../routes/taskRoutes");
9 const app = express();
10 connectDB();
11
12 app.use(express.json());
13 app.use(cors());
14 app.use(morgan("dev"));
15
16 app.get("/", (req, res) => {
17   console.log("Root route hit");
18   res.send("Task Management API is running...");
19 });
20
21
22 const PORT = process.env.PORT || 3000;
23
24 app.use("/api/tasks", taskRoutes);
25 app.listen(PORT, () => {
26   console.log(`Server running on port ${PORT}`);
27 });
```


← → | Alfredo mern-stack

EXPLORER

2026-06-13 at 10.01.50 PM.png JS taskRoutes.js X .env .env.example JS server.js GET http://loca...

OPEN EDITORS

JS db.js config

JS taskController.js c...

JS Task.js models

Screenshot 2026-...

JS taskRoutes.js routes

.env

.env.example

JS server.js

GET http://loca...

Extension: Postman

ALFID...

config

JS db.js

controllers

JS taskController.js

models

JS Task.js

node_modules

routes

JS taskRoutes.js

.env

.env.example

package-lock.json

package.json

Screenshot 2026-...

JS server.js

OUTLINE

TIMELINE

```
1 const express = require("express");
2 const router = express.Router();
3
4 const {
5   createTask,
6   getTasks,
7   getTaskById,
8   updateTask,
9   deleteTask,
10 } = require("../controllers/taskController");
11
12 router.post("/", createTask);
13 router.get("/", getTasks);
14 router.get("/:id", getTaskById);
15 router.put("/:id", updateTask);
16 router.delete("/:id", deleteTask);
17
18 module.exports = router;
```

Ln 18, Col 25 Spaces: 4 UTF-8 LF JavaScript Finish Setup Go Live Prettier

EXPLORER

1.50 PM.png JS taskRoutes.js .env .env.example JS server.js GET http://loca... Ex

OPEN EDIT... .env

```
1 PORT=3000
2
3 MONGO_URI=mongodb+srv://admin:Admin123456@cluster0.ubbjmvw.mongodb.net/?appName=Cluster0
```

JS db.js config

JS taskController.js c...

JS Task.js models

Screenshot 2026-...

JS taskRoutes.js routes

.env

.env.example

JS server.js

GET http://loca...

